Speaker Guide: MCP Hallucination Checker (deck v9, 27 slides)

Use with `MCP_Hallucination_Checker_Review_v9.pptx`. For each slide: what to say, what to point at, abbreviations to define, and how to defend it.
Target length: about 15 minutes for slides 1 to 25, plus questions. Slides 26 and 27 are references and are not presented.

Before you start

30-second pitch (memorise this)
"AI agents now take real actions through tools. If an agent's reasoning slips, for example turning $50 into $500 or dropping a 'not', the mistake becomes a real action. I built a checker that runs before the tool executes. It validates the arguments against the tool's schema, and it checks the agent's reasoning against the source text using graph comparison and a language-inference model. I tested each part, and the results show which parts work and what I am improving next."

Your three honest messages

1. The deterministic schema check works and is fast.
2. Windowed NLI is the strongest semantic signal: on the full public benchmark it raised AUROC from 0.672 to 0.745 against plain truncation (same model, same claims, paired CI +0.068 to +0.077).
3. My ablation showed the graph stage does not yet help, so I know what to rework.

Never say: "first", "novel method", "state of the art", "production ready", "solves hallucination", "100% accurate" (without saying it is the schema check on constructed cases).

If you do not know an answer: "I have not measured that. It is on my planned list, and here is how I would test it."

Slide 1: Title

Say (20 s): Greet, give the title, say it is a project review, and state it is a research prototype.
Explain: "Self-reflective" means the agent checks its own plan before acting. "Via MCP" means the checker is delivered as an MCP server so any compatible agent can use it.
Abbreviations: MCP = Model Context Protocol, a standard for connecting AI agents to tools. LLM = large language model.
Defend: None needed.

Slide 2: Background and Motivation

Say (60 s): Agents increasingly act through tools: payments, tickets, deployments, databases. The call is built from the agent's own reasoning, so an error in the reasoning becomes an action. Three examples: `$50` becomes `$500` (a value error), a dropped "not" turns "do not delete the backup" into a delete (negation), and a missing required field makes the action fail or misroute (schema).
Explain: These are small textual changes with large real consequences, which is why they matter more than ordinary text errors.
Abbreviations: MCP [11]; "tool-call boundary" = the moment between the agent deciding and the tool running.
Likely questions

- *Is this a real problem or a toy example?* The examples are representative of failure types, not measurements of real agent incidents. I did not measure how often agents make them; the literature I cite (tool-hallucination work [13], [14], [15]) shows these failures occur in real models.
- *Don't modern LLMs already avoid this?* They reduce it but do not remove it. Evidence from the papers: the Closed-World Resolution paper [13] found hallucinations even in frontier models and that model size did not

help; AgentHallu [12] found the best model located the hallucinating step only 41.1% of the time. Structured-output modes guarantee valid format, not correct values.

Slide 3: Problem Statement

Say (45 s): Given the agent's reasoning, the source it should be grounded in, and its planned arguments with the tool schema, decide before execution whether the call is schema-valid and factually consistent, and point to the unsupported span. Three difficulties: values differ by a character, meaning flips while words stay, and self-checks are optional.
Point at: the Inputs → Gate → Outputs diagram.
Abbreviations: hypothesis = the claim being checked (agent's reasoning); premise = the source it should be grounded in; span = a piece of text.
Likely questions

- *Why "before execution"?* After-the-fact detection cannot undo a payment or a deletion.
- *Where does the source context come from?* From the application, such as the user's request, a document or a database record. Without it, only the schema check can run. State this limitation openly.

Slide 4: Existing System / Literature Review

Say (75 s): Three families. (1) Graph-based factuality: FactGraph [1], AMRFact [2], GraphEval [9], and the 2005 graph-matching entailment work [10]. They compare semantic structure but target summaries or general text, not tool calls. (2) NLI and grounding checkers: SummaC [5], AlignScore [6], MiniCheck [3] with the LLM-AggreFact benchmark [4]. They check whether a claim is supported by a document. (3) Tool-call hallucination and MCP guardrails: Closed-World Resolution [13] checks that tools and arguments exist; SAAG [14] checks schema and then exact-match argument grounding; Healy et al. [15] detect tool-selection errors from model internals; AgentHallu [12] locates the faulty step; MCP proxies [16], [17] do schema, policy and approval checks.
Abbreviations: AMR = Abstract Meaning Representation (a semantic graph format). NLI = natural language inference (entails / contradicts / neutral). NAACL, EMNLP, ACL, KDD = conferences. arXiv = preprint server; citations marked arXiv are preprints and not all are peer-reviewed.
Likely questions

- *Did you read all of these?* Yes, the abstracts and the relevant method sections for the papers I cite. I dropped anything I could not verify. (This is true for the reference list as built; re-check before presenting.)
- *Why not use MiniCheck directly?* It is a strong baseline and I ran it (slide 24): it beats my windowed DeBERTa on grounding. It checks text against a document but does not do schema validation or tiered gating.

Slide 5: Research Gap

Say (60 s): Read the matrix: schema/policy proxies validate arguments but do not check content against a source; hallucination resolvers check tool existence; graph and NLI checkers work on text but are not gates for tool calls. To our knowledge, in the literature we reviewed, no evaluated system combines schema validation, value-strict structural graph matching and span-level NLI in a risk-tiered pre-execution gate. Each ingredient exists. The open question is whether the combination helps, and at what latency and false-positive cost.
Explain: The last row is a claim about my prototype's *design*, not about its performance.
Likely questions

- *So is it novel?* The components are not; the combination at the tool-call boundary is the contribution to examine. My own ablation shows the combination does not yet beat NLI alone, so I claim the system and the evidence, not a performance advantage.
- *How do you know no one did it?* I do not. My search was limited, so I say "in the literature we reviewed."
- *"Yes/No" cells look generous to your row.* The cells describe which capabilities each class includes, not quality. My row says "Yes" for all four because the system implements all four; the ablation tells you how well they work.

Slide 6: Research Objectives

Say (45 s): Six measurable objectives: detect value mutations, negation flips and role swaps, schema violations; localise unsupported spans; enforce through a proxy; evaluate openly with ablations.
Be ready to say which are met: O3 (schema) met on my cases; O1/O2 partly met (content recall only 0.25, negation 60%, role swap 40% on the full pipeline); O4 met (spans and types returned); O5 implemented, not evaluated end to end; O6 in progress.
Defend: "Objectives are measurable so I can report where I fall short, and I do."

Slide 7: Dataset / Experimental Setup

Say (60 s): Two datasets. MCPEvol-Bench: 60 hand-written cases (29 consistent, 31 hallucinated) across content, format, schema, negation, role swap; a smoke test I built. LLM-AggreFact: public benchmark, 29,320 test claims, 77.9% supported, so I report balanced accuracy and AUROC. Models: DeBERTa-v3-base NLI, BERT-NER, spaCy. Thresholds: GED 0.35, NLI 0.40 for the internal set; the external threshold chosen on the dev split. Windows: 180 words, 40 overlap. Runtime: internal on CPU, full external run on a Kaggle T4 GPU.
Abbreviations: GED = graph edit distance. AUROC = area under the ROC curve. CC-BY-ND = Creative Commons licence (no derivatives). T4 = a data-centre GPU.
Likely questions

- *Is 60 cases enough?* No; it is a smoke test, which is why I add ablations and an external dataset and plan a larger benchmark.
- *Did you tune thresholds on the test data?* No. Internal thresholds were fixed (0.35, 0.40) and not tuned on these cases in the reported runs; the external threshold came from the dev split and was applied once to test.
- *Does AggreFact test tool calls?* No. It tests grounding of claims in documents, so it measures the NLI component's general ability, not tool-call hallucination.

Slide 8: Methodology: Overall Functionality

Say (45 s): Walk left to right: the agent sends a tool call to the proxy gate (Module 6). The gate applies the risk tier and invokes the two MCP tools: schema alignment (Module 4) and the logic chain (Modules 1, 2, 3, 5). Output is PASS (tool executes) or BLOCK (typed spans and feedback returned to the agent).
Point at: the dashed box (the MCP server) and the two output boxes.
Defend: "The diagram is drawn as editable shapes; each box maps to a file in `server/`."

Slide 9: Methodology: Workflow

Say (45 s): Eight steps: inputs, build graphs, value-aware GED, structural verdict, span NLI, taxonomy, constraints and drift, decision. Emphasise the final rule: a structural pass can still be overridden by a low-entailment span or a missing constraint.
Likely question: *Why can NLI override GED?* The design chose a conservative gate: a call should be blocked if any span is unsupported. The ablation shows this conservatism, with the structural verdict noisy, is part of why the hybrid underperforms NLI alone.

Slide 10: Module 1: Semantic Frame Extraction

Say (60 s): Text goes through spaCy parsing, BERT-NER for names and spaCy NER for values, then the governing verb of each entity is found to form frames. Dependency labels become roles: subject = AGENT, object = PATIENT, indirect object = RECIPIENT. A `neg` child marks the verb as negated. I call this "SRL-style" honestly: it is NER plus dependency heuristics, not a trained SRL model.
Abbreviations: NER = named entity recognition. SRL = semantic role labelling. POS = part of speech.
Likely questions

- *Why not a real SRL model?* To avoid a heavy dependency (AllenNLP) and keep it light. The cost is accuracy; a trained SRL model is a planned improvement.

- *Known weakness?* Two entities of the same type under one verb collide (only the first is kept), so role-swap detection relies on the separate dependency edges.

Slide 11: Module 2: Graph Construction and Value-Aware GED

Say (75 s): Build a graph for the hypothesis and one for the premise. GED is the cheapest set of edits turning one into the other. My cost model makes any change to a number, date or version cost the maximum (1.0), and a negation flip also costs 1.0. The Hungarian algorithm finds the best node pairing. The distance is normalised and a +0.20 bonus is added per unmatched value node so numeric changes cannot hide in big graphs. Above 80 nodes a faster approximation is used. Threshold 0.35.
Abbreviations: GED = graph edit distance. Hungarian algorithm = optimal assignment solver (`scipy.linear_sum_assignment`). Node/edge = graph elements.
Likely questions

- *Why did GED hurt in your ablation?* My explanation is a design hypothesis: the distance is symmetric, so extra premise content raises the distance, and the value bonus penalises premise-only numbers. I tested one fix (asymmetric variant), which raised the hybrid from 57.1% to 62.9%, not significant at n = 35.
- *Why keep it then?* It is explainable, catches small value changes, and I am reworking it into a hypothesis-side check. I do not claim it currently improves accuracy.
- *Complexity?* The Hungarian step is cubic in graph size, hence fast mode.

Slide 12: Module 3: Span-Level NLI with Windowing and Taxonomy

Say (75 s): For each span I ask a DeBERTa NLI model whether the premise supports it. Long premises are split into 180-word windows with 40-word overlap, each window is scored, and the maximum is kept. Previously, long documents were truncated at 512 tokens and later evidence was lost. Spans scoring below 0.40 are flagged and labelled schema, format or content.
Abbreviations: NLI = natural language inference. DeBERTa = a Transformer encoder architecture. Token = word piece (limit 512). Taxonomy = the three labels.
Likely questions

- *Why windowing?* The model has a 512-token limit. Windowing raised AUROC from 0.672 to 0.745 on the full test split in the like-for-like comparison.
- *Why max over windows?* Evidence for a claim usually sits in one place; the max finds it. The cost is that a loosely related window could occasionally inflate a score.
- *Why DeBERTa-base, not a bigger model?* Speed and CPU-friendliness; a stronger checker such as MiniCheck is part of my comparison.
- *Template bug?* The date template matches via a substring test, so most spans get the date wording. On a 2,000-claim sample, raw versus templated changed AUROC little (0.753 vs 0.749).

Slide 13: Module 4: Schema Alignment Validator

Say (45 s): Deterministic validation: required fields (schema mismatch), then type, enum, pattern, range and length (format mismatch). Example: the benchmark case where `quantity` and `address` are missing, so the call is blocked.
Abbreviations: JSON Schema = a standard way to describe valid JSON data.
Likely questions

- *100% accuracy, really?* Yes, on 25 constructed cases, which the validator is designed to catch. It shows the validator works, not that it handles semantic errors.
- *What does it miss?* Schema-valid but wrong values. That is why the logic check exists.

Slide 14: Module 5: Constraints and Session Drift

Say (45 s): If the user's original request is supplied, extract its names, numbers and times and check that the hypothesis preserves them. Separately, a rolling mean of the last 5 consistency scores per session raises a drift alert below 0.60; sessions are capped and expire after an hour.

Abbreviations: LRU = least recently used (eviction). TTL = time to live.
Likely questions

- *How were 0.60 and window 5 chosen?* They are untuned defaults; I flag it as future calibration.
- *Weakness of constraints?* It is substring matching, so paraphrases ("fifty" vs "50") can be false alarms.

Slide 15: Module 6: Proxy Gate with Risk Tiers

Say (60 s): LOW passes the call, MEDIUM runs the schema check, HIGH runs schema then logic. A failed check returns a structured block with the reason instead of calling the tool. Point at the scope note: the HIGH-tier logic check runs only when both reasoning and source context are supplied; otherwise the schema check applies.
Likely questions

- *Can the agent bypass the checker?* Only calls routed through the proxy are enforced. The decorator by itself does not enforce; I call this a gate when routed through the proxy.
- *Who assigns risk tiers?* The developer, manually. Mislabelling reduces protection.
- *Fail-open or fail-closed?* Currently fail-open when context is missing; a fail-closed policy is a planned option.
- *Latency?* I have not benchmarked end-to-end latency; the NLI step dominates on CPU.

Slide 16: Methodology: Algorithms

Say (45 s): Algorithm 1 is the node cost: exact match 0, attribute drift 0.3, negation flip 1.0, value mismatch 1.0, same kind 0.5, otherwise 1.0. Algorithm 2 is the verification decision: build graphs, GED, value bonus, flag spans below the NLI threshold, add missing constraints, record session score, return the result.
Defend: "The costs are design choices, not learned; I did not tune them, and ablations on weights are future work."

Slide 17: Methodology: Grounded Mode (redesign)

Say (60 s): After the ablation I added a second verification mode. A claim is consistent only if three things hold: the whole claim is entailed by the source (same windowed DeBERTa, no span template, threshold 0.40); every number, date, version or time in the plan also appears in the source (a deterministic check that catches near-identical numbers such as 3225 versus 3200, which NLI sometimes accepts); and, if the user's request is supplied, its key facts are preserved. The offending value becomes the reported mismatched span. The original hybrid mode is still available as an option.
Point at: the three parallel checks feeding the decision box.
Abbreviations: NLI = natural language inference. Entailment threshold 0.40 = the probability above which the source is taken to support the claim.
Likely questions

- *Why drop the graph stage?* In my ablation the graph distance alone was close to chance (AUROC about 0.52) and the hybrid scored about 53% on held-out cases, while whole-claim NLI scored 94%. I kept the hybrid as an option and am reworking it.
- *Is the value check just a regex?* Yes, deliberately: it is deterministic, explainable, and names the offending token. It does not handle spelled-out numbers or unit conversions.
- *What does it get wrong?* Plans phrased as future actions ("The agent will download X as requested") can be judged not entailed by a request-style source, and localisation is coarser than span-level NLI.

Slide 18: Methodology: Argument-Level Grounding

Say (60 s): A tool call is a set of typed values, so I check each argument against the conversation it should come from. Numbers are matched after normalisation, including spelled-out numbers and percentages; ISO dates and timestamps are compared with written dates and clock times; currency codes are matched to currency names; other strings use normalised, stemmed and fuzzy matching. The call score is the weakest argument, and the call is blocked below a threshold tau. I calibrate tau on genuine calls with a split-conformal quantile so the false-block rate stays within a chosen budget for each risk tier.
Abbreviations: ISO = the international date/time format (2022-03-15T14:00:00). Conformal = a method giving a

finite-sample guarantee on a rate. Tau / alpha = the score threshold / the false-block budget.

Likely questions

- *How is this different from exact-match argument grounding?* Exact matching compares literal text, so reformatted values ("2 pm" vs a timestamp, "JPY" vs "Japanese yen", "twelve" vs 12) are false blocks. I normalise by type. I implemented the exact-match baseline myself, so the comparison is against my reading of that idea, not the original system.
- *What does the guarantee mean?* If genuine calls at deployment look like the calibration calls, the chance a genuine call is blocked is at most alpha. It does not cover distribution shift, and it says nothing about detection.
- *What can't it do?* World knowledge (a ticker symbol for a company name), arithmetic, booleans, and non-English text.

Slide 19: Preliminary Results: Internal

Say (60 s): Table 1: 60 cases. Schema and format 100% on 25 constructed violations. Text cases are weaker: content 60% (recall 0.25), negation 60%, role swap 40%. Overall 75.0%, precision 0.82, recall 0.62 with "consistent" as the positive class.

Abbreviations: Precision = of those called consistent, how many truly are. Recall = of truly consistent cases, how many were caught. Positive class = the class counted as "positive" in metrics.

Likely questions

- *Why did the older documents say 83.3%?* That figure came from an earlier run I could not tie to a saved result file, so I use the latest saved run (75.0%).
- *Is it overfitted?* Thresholds were not tuned on these cases in the reported runs. With only 60 cases, treat it as a smoke test.

Slide 20: Preliminary Results: Ablation

Say (75 s): Ablation on the 35 text-based cases at fixed thresholds. NLI-only 80.0% with precision 1.00; GED-only 54.3%; current hybrid 57.1%; hybrid with an experimental asymmetric GED 62.9%. An exact McNemar test shows NLI-only beats the current hybrid ($p = 0.008$). The asymmetric gain is not significant at $n = 35$. Conclusion: NLI grounding is the strongest signal, and the graph stage needs rework.

Abbreviations: Ablation = remove or swap a component to measure its contribution. McNemar = paired test on cases where two systems disagree. p-value = probability of a difference this large if there were no real difference. Specificity = of truly hallucinated cases, how many were caught. Balanced accuracy = mean of recall and specificity.

Likely questions

- *Doesn't this undermine your project?* It shows I tested my own design, and it points to the next improvement. The system still delivers the schema gate and a strong NLI check; the structural stage is the part I am reworking.
- *Only 35 cases; is it reliable?* The McNemar result (0 cases where hybrid is right and NLI-only wrong, 8 the reverse) is strong for this set, but the set is small and written by me. I say so and plan a larger one.
- *Did you pick the best variant after seeing the data?* The four variants and thresholds were fixed beforehand; nothing was tuned on these cases. The asymmetric variant is labelled experimental.

Slide 21: Preliminary Results: Redesign

Say (75 s): Table 5 compares four variants on three sets of text-based cases at fixed thresholds. On the held-out set of 214 cases the shipped hybrid scores 52.8%, NLI with the date template 76.6%, raw whole-claim NLI 94.4%, and the grounded mode 95.3%. The dev set is my original 35 cases. The design set is a generated set of 210 cases whose errors motivated the value check, so it is optimistic. The held-out set is a second generated set drawn with a different seed and not inspected before the design. On the held-out set the value check fixes two cases and breaks none, but that is not statistically significant. Graph distance alone is near chance (AUROC about 0.52).

Abbreviations: Dev / design / held-out = tuning-style, design-informing and clean test sets. McNemar = paired test on disagreements. AUROC = ranking quality (0.5 chance).

Likely questions

- *Isn't this circular? You built the cases and the checker.* Partly. The cases are generated from templates with labels correct by construction, and the held-out set uses the same templates, so it tests new draws, not new kinds of cases. That is why I call it a larger smoke test, and why real tool-call traces and an external benchmark are my next step.
- *Why is the shipped hybrid so low?* The graph stage over-flags consistent cases (symmetric distance penalises extra source content) and the combination rule lets any weak span block a call; the ablation measured this, and the grounded mode is the response.
- *Is the 95.3% your headline?* No. It is a held-out result on synthetic cases; the external AggreFact results are the more honest guide to general grounding ability, and those are lower.

Slide 22: Preliminary Results: Argument Grounding

Say (75 s): I built ArgGround-Bench from the public Glaive function-calling dataset: each case is a genuine call with the conversation before it, or the same call with one argument value replaced by a value absent from the conversation. These conversations are LLM-generated, not real deployments. On the held-out half of 1,751 cases, exact string matching blocks 14.0% of genuine calls; type-aware matching blocks 7.3% at the same detection of about 96.5%. A calibrated 5% budget gave 3.7% false blocks with 87.5% detection, and a 10% budget gave 7.3% with 97.8%. A 2% budget is not achievable at this stage because about 7% of genuine calls contain values the conversation never states.
Abbreviations: False-block rate = share of genuine calls wrongly blocked. Detection = share of injected errors blocked. Budget = the maximum false-block rate you ask for.
Likely questions

- *Are these real agent errors?* No. The errors are synthetic single-argument substitutions in LLM-generated conversations. Natural model-made errors and a human-checked subset are the next step.
- *Why did you run the test half twice?* The first run used a version with a regex bug (backspace characters instead of word boundaries) that a unit test exposed; I fixed it and re-ran. The fix was not tuned on test results.
- *What about the remaining 7% false blocks?* Mostly world knowledge (ticker symbols), model-chosen defaults and derived dates. A DeBERTa NLI second stage did not help on the dev half; a stronger model or LLM judge is the next experiment.

Slide 23: Preliminary Results: External (full test split)

Say (75 s): Table 3: the NLI component alone on all 29,320 claims, on a GPU. The clean comparison is truncation versus windowed with the same raw claims and a dev-chosen threshold for each: AUROC 0.672 to 0.745 (paired difference +0.0725, 95% CI 0.068 to 0.077) and balanced accuracy 0.616 to 0.673. My earlier CPU run (templated claims, fixed 0.40) had AUROC 0.675 and is shown for reference only. At the fixed 0.40 threshold windowed balanced accuracy is 0.636. A seeded 500-claim laptop sample gives 0.740, consistent. Because 77.9% of claims are supported, always answering "supported" gives 77.9% accuracy but 0.5 balanced accuracy, so I report AUROC and balanced accuracy.
Abbreviations: AUROC = probability that a supported claim is scored above an unsupported one (0.5 chance, 1.0 perfect). CI = confidence interval. Dev/test = tuning split / final split.
Likely questions

- *Why is the calibrated threshold 0.005?* Model scores are very peaked, so a very low cut-off maximises balanced accuracy. I rely on AUROC as the robust figure.
- *Why is accuracy low (49%) at 0.40?* The class imbalance and high-precision behaviour: at 0.40 it rejects many supported claims (recall 0.38). That is why I do not use raw accuracy.
- *Does this prove your pipeline works?* No. It evaluates the NLI component only, not GED or the gate, and not tool calls.
- *How does it compare to MiniCheck?* See slide 24: on the same 5,000 claims MiniCheck gets AUROC 0.821 against 0.744 for my windowed DeBERTa, a clear gap, at about 14 times the cost per claim. Published MiniCheck figures should not be compared directly to mine without running it on the same data.

Slide 24: Preliminary Results: Baseline Comparison

Say (60 s): I compared my NLI component with a published fact-checker, MiniCheck flan-t5-large, on the same 5,000 seeded test claims, each system's threshold tuned on 3,000 dev claims. MiniCheck reaches AUROC 0.821, my windowed DeBERTa 0.744, plain truncation 0.674. So windowing closes about half of the gap and the remaining gap is clear (paired AUROC difference about 0.078). The trade-off is speed: about 0.40 seconds per claim for MiniCheck versus 0.028 for DeBERTa on a T4, roughly 14 times slower. My plan is to test MiniCheck as the grounding model inside the pipeline, with DeBERTa as a fast first pass.
Abbreviations: Baseline = a reference system to compare against. Paired bootstrap = resampling the same claims to put a confidence interval on the difference between two systems.
Likely questions

- *Why only 5,000 claims?* MiniCheck is about 14 times slower; the full split would take around 8 hours on a T4, and my first full-split attempt ran out of GPU memory. The subset is seeded and stratified, and all three systems are scored on exactly the same rows.
- *So why not just use MiniCheck?* It is the better grounding model, but it is slower per claim and it does not do schema validation, tiering or span localisation. Using it inside the gate is my next experiment.
- *Is this a result for your whole system?* No. All three are fact-check scorers; GED is not involved.

Slide 25: Preliminary Results: AUROC by Source

Say (45 s): Strongest on Reveal (0.92), Lfqa (0.82), FactCheck-GPT (0.80); weakest on ExpertQA (0.59), AggreFact-CNN (0.62) and RAGTruth (0.66). RAGTruth is 56% of the test split, so it pulls the overall number down. Retrieval-augmented outputs and expert answers are the hard cases.
Likely question: *What would help on the weak sources?* A stronger checker and possibly domain-specific structure; I would test MiniCheck and an LLM judge first.

Slides 26 and 27: References

Do not read them. If asked about a source, say which of [1] to [18] it is and what it contributes (see Slide 4). Be ready to say you read the abstract and method sections, and that GitHub repositories [16], [17] are software projects cited with an access date.

Closing (30 s)

"In summary: I built a pre-execution checker for tool calls with a deterministic schema gate and a windowed NLI grounding check. Testing each part showed that windowing fixed long-document detection and that my graph stage needs rework, so my next steps are the hypothesis-side graph check, a MiniCheck and LLM-judge comparison, and a larger tool-call benchmark."

Hard questions and how to answer

Question	Honest answer
Is this novel?	The components are not. The combination at the MCP tool-call boundary is the contribution to examine, and my ablation says the combination does not yet outperform NLI alone, so I claim a working system with measured evidence.
Why not ask an LLM to verify?	I have no LLM-judge baseline yet. Advantages of my approach are determinism for schema checks, low cost and latency with a local model, independence from the agent, and span localisation. Adding an LLM-judge baseline is a planned step.
Does it work on real agent traces?	Not tested. Evaluation used my hand-written cases and a public grounding benchmark.
What is your biggest weakness?	The graph stage does not help yet, my internal benchmark is small and self-written, and the gate fails open without context.

Question	Honest answer
What did you actually build versus reuse?	Reused: spaCy, BERT-NER, DeBERTa NLI, NetworkX, scipy, FastMCP. Built: the pipeline, cost model, proxy gate, validator, session tracker, benchmark and evaluation tooling.
How do I reproduce your numbers?	`make bench`; `PYTHONPATH=. python benchmarks/ablation_local.py`; `PYTHONPATH=. python benchmarks/aggrefact_sample.py --n 500 --seed 42`; the full run is the Kaggle notebook.
Why are AggreFact results relevant to tool calls?	They test whether the NLI component can detect unsupported claims in context, a capability the tool-call check relies on; they are not tool-call results.
Are your thresholds tuned?	Internal: fixed at 0.35 and 0.40, not tuned in the reported runs. External: tuned on the dev split once, applied once to test.

Abbreviation cheat-sheet

AMR: Abstract Meaning Representation. AUPRC: area under precision-recall curve. AUROC: area under ROC curve. CI: confidence interval. CNN/XSum: summarisation datasets inside AggreFact. CPU/GPU: processors. DeBERTa: Decoding-enhanced BERT with disentangled attention. dev/test: tuning/final data splits. F1: harmonic mean of precision and recall. FN/FP/TN/TP: false negative/positive, true negative/positive. GED: graph edit distance. JSON: JavaScript Object Notation. LLM: large language model. LRU: least recently used. MCP: Model Context Protocol. NER: named entity recognition. NLI: natural language inference. RAG: retrieval-augmented generation (RAGTruth is a RAG hallucination dataset). SRL: semantic role labelling. TTL: time to live.